

CO4 ASSESSMENT TOOL 2 – CASE STUDY

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
C PROGRAMMING

RESHINI PRIYA B
1012001558

CO4 - AT2 - CASE STUDY

QUESTIONS

Q1
CO4 - AT2 - CASE STUDY
25 marks

Q2
CO4 - AT2 - CASE STUDY
25 marks

Q3
CO4 - AT2 - CASE STUDY
25 marks

Q4
CO4 - AT2 - CASE STUDY
25 marks

4/4 answered

Q1 CO4 - AT2 - CASE STUDY

25 marks

A large enterprise resource planning application is built from multiple source files, including modules for sales, inventory, and finance, and all of these modules must reference a common variable that stores the current fiscal year. The variable is defined once in one source file but must be read and, in some cases, modified by functions compiled separately in other source files. Without proper declaration, each module would either fail to compile or maintain its own inconsistent copy of the fiscal year. The design must ensure a single, consistent value is shared across the entire application.

Question: Identify which storage class allows the fiscal-year variable to be accessed across multiple source files, and explain the role of extern in achieving this shared access.

Your Code

```
1 #include <stdio.h>
2 int fiscal_year = 2026;
3
4 void salesModule()
5 {
6     printf("Sales Module - Fiscal Year: %d\n", fiscal_year);
7 }
8 void inventoryModule()
9 {
10    printf("Inventory Module - Fiscal Year: %d\n", fiscal_year);
11 }
12 void financeModule()
13 {
14     fiscal_year = 2027;
15     printf("Finance Module - Fiscal Year Updated to: %d\n", fiscal_year);
16 }
17 int main()
18 {
19     salesModule();
20     inventoryModule();
21     financeModule();
22
23     salesModule();
24     inventoryModule();
25
26     return 0;
27 }
```

Input

stdin Input

Output

Run

Save

© 2026 SIMATS Online Lab. All rights reserved.

C04 - AT2 - CASE STUDY

← Back

QUESTIONS

Q1
C04 - AT2 - CASE STUDY
25 marksQ2
C04 - AT2 - CASE STUDY
25 marksQ3
C04 - AT2 - CASE STUDY
25 marksQ4
C04 - AT2 - CASE STUDY
25 marks

4/4 answered

C04 - AT2 - CASE STUDY

25 marks

A smart utility meter installed in a residential complex can measure electricity consumption, water flow, or gas usage, but the hardware reports only one utility type per polling cycle depending on which sensor is currently active. Common information such as meter ID, timestamp, and location must always be recorded alongside whichever utility reading is captured. The billing server receives thousands of polling records daily and must interpret the correct reading type each time. Minimizing per-record memory footprint is a key design goal for the embedded firmware.

Question: Design a structure that embeds a union for the utility-specific reading, and explain why this hybrid representation is more memory-efficient than using three separate full-sized fields.

Your Code

```
#include <stdio.h>
#include <string.h>

enum {
    ELECTRICITY,
    WATER,
    GAS
} UtilityType;

typedef union {
    float electricityUnits; // in kWh
    float waterLiters;      // in liter
    float gasCubicMeters;   // in cubic meters
} UtilityReading;

typedef struct {
    int meterID;
    char timestamp[20];
    char location[50];
    UtilityType type; // tells us which union member is valid
    UtilityReading reading; // union: only one member used per record
} MeterRecord;

void displayRecord(MeterRecord record) {
    printf("Meter ID : %d\n", record.meterID);
    printf("Timestamp : %s\n", record.timestamp);
    printf("Location : %s\n", record.location);

    switch (record.type) {
        case ELECTRICITY:
            printf("Type : Electricity\n");
            printf("Reading : %.2f kWh\n", record.reading.electricityUnits);
            break;
        case WATER:
            printf("Type : Water\n");
            printf("Reading : %.2f litera\n", record.reading.waterLiters);
            break;
        case GAS:
            printf("Type : Gas\n");
            printf("Reading : %.2f Cubic Meters\n", record.reading.gasCubicMeters);
            break;
    }
}

int main() {
    MeterRecord record1, record2, record3;

    // Electricity reading
    record1.meterID = 101;
    strcpy(record1.timestamp, "2025-04-05 10:00");
    strcpy(record1.location, "Block A - Flat 12");
    record1.type = ELECTRICITY;
    record1.reading.electricityUnits = 245.75;

    // Water reading
    record2.meterID = 102;
    strcpy(record2.timestamp, "2025-04-05 10:05");
    strcpy(record2.location, "Block B - Flat 4");
    record2.type = WATER;
    record2.reading.waterLiters = 1520.50;

    // Gas reading
    record3.meterID = 103;
    strcpy(record3.timestamp, "2025-04-05 10:10");
    strcpy(record3.location, "Block C - Flat 9");
    record3.type = GAS;
    record3.reading.gasCubicMeters = 12.35;

    printf("=== Smart Utility Meter Polling Records ===\n");
    displayRecord(record1);
    displayRecord(record2);
    displayRecord(record3);

    printf("\nSize of MeterRecord (struct with union) : %d bytes\n",
           sizeof(MeterRecord));
    printf("Size of UtilityReading (union alone) : %d bytes\n",
           sizeof(UtilityReading));

    return 0;
}
```

Run

Save

Input

stdin input

Output

CO4 - AT2 - CASE STUDY

← Back

QUESTIONS

Q1
CO4 - AT2 - CASE STUDY
25 marksQ2
CO4 - AT2 - CASE STUDY
25 marksQ3
CO4 - AT2 - CASE STUDY
25 marksQ4
CO4 - AT2 - CASE STUDY
25 marks

4/4 answered

Q3 CO4 - AT2 - CASE STUDY

25 marks

An insurance company processes claims for health, vehicle, and property policies, where each claim type requires distinct supporting information such as hospital details, accident details, or property-damage details respectively. Every claim, regardless of type, shares common fields such as claim ID, policyholder name, and claim date. Claims officers repeatedly retrieve and update a claim's status as it moves through approval stages. The company wants a design that avoids wasting memory on fields that do not apply to a given claim type. Question: Design a structure containing a union to represent claim-type-specific information, and explain how a structure pointer can be used to update a claim's status efficiently as it progresses through the workflow.

Your Code

```
#include <stdio.h>
#include <string.h>

enum ClaimType { HEALTH, VEHICLE, PROPERTY };

union ClaimDetails {
    struct {
        char hospitalName[50];
        float billAmount;
    } health;
    struct {
        char vehicleNumber[20];
        char accidentLocation[50];
    } vehicle;
    struct {
        char propertyAddress[50];
        float damageCost;
    } property;
};

struct Claim {
    int claimID;
    char policyHolderName[50];
    char claimDate[15];
    enum ClaimType type;
    char status[20];
    union ClaimDetails details;
};

void updateStatus(struct Claim *c, const char *newStatus) {
    strcpy(c->status, newStatus); // operation used for efficiency
    printf("Claim ID %d status updated to: %s\n", c->claimID, c->status);
}

int main() {
    struct Claim c1;

    // Common fields
    c1.claimID = 101;
    strcpy(c1.policyHolderName, "Reshini Priya");
    strcpy(c1.claimDate, "2025-09-01");
    strcpy(c1.status, "Submitted");
    c1.type = HEALTH;

    // Type-specific fields (only health details are set)
    strcpy(c1.details.health.hospitalName, "Apollo Hospital");
    c1.details.health.billAmount = 25000.50;

    // Display initial claim
    printf("Claim ID: %d\n", c1.claimID);
    printf("Policy Holder: %s\n", c1.policyHolderName);
    printf("Claim Date: %s\n", c1.claimDate);
    printf("Initial Status: %s\n", c1.status);
    printf("Hospital: %s, Bill: %.2f\n",
           c1.details.health.hospitalName, c1.details.health.billAmount);

    // Update status through workflow stages using pointer
    struct Claim *ptr = &c1;
    updateStatus(ptr, "Under Review");
    updateStatus(ptr, "Approved");
    updateStatus(ptr, "Settled");

    return 0;
}
```

Run

Save

Input

stdin input

Output

C04 - AT2 - CASE STUDY

← Back

QUESTIONS

Q1
C04 - AT2 - CASE STUDY
25 marksQ2
C04 - AT2 - CASE STUDY
25 marksQ3
C04 - AT2 - CASE STUDY
25 marksQ4
C04 - AT2 - CASE STUDY
25 marks

4/4 answered

Q4 C04 - AT2 - CASE STUDY

25 marks

A distribution warehouse maintains inventory records for tens of thousands of products, where each record stores a product code, product name, quantity in stock, and supplier address consisting of several related sub-fields. Warehouse staff frequently search for a specific product to check stock levels and update the quantity after every incoming or outgoing shipment. Because the inventory is large, copying entire records during routine updates would be inefficient. The system must remain responsive even during high-volume shipment processing.

Question: Design an array of structures to hold the inventory, and explain how a structure pointer allows staff to update a selected product's stock quantity without copying the complete record.

Your Code

```
#include <stdio.h>
#include <string.h>

#define MAX_PRODUCTS 5

// Structure for supplier address with related sub-fields
struct SupplierAddress {
    char city[10];
    char state[10];
    char pincode[10];
};

// Structure for a single inventory record
struct Product {
    int productCode;
    char productName[50];
    int quantity;
    struct SupplierAddress supplier;
};

// Function to update stock using a structure pointer
// No copying of the entire record occurs; only the
// quantity field of the original record is modified
// through the pointer (pass by reference).
void updateStock(struct Product *p, int changeInQty) {
    p->quantity += changeInQty;
}

// Function to search a product by product code
// Returns pointer to the matching record (or NULL)
struct Product* searchProduct(struct Product inventory[], int n, int code) {
    for (int i = 0; i < n; i++) {
        if (inventory[i].productCode == code) {
            return &inventory[i];
        }
    }
    return NULL;
}

int main() {
    // Array of structures to hold inventory
    struct Product inventory[MAX_PRODUCTS] = {
        {101, "RED Bull", 500, {"Chennai", "Tamil Nadu", "600001"}},
        {102, "Steel Pipe", 200, {"Coimbatore", "Tamil Nadu", "641001"}},
        {103, "Cotton Fabric", 750, {"Salem", "Tamil Nadu", "636001"}},
        {104, "Plastic Container", 1000, {"Madurai", "Tamil Nadu", "625001"}},
        {105, "Wooden Pallet", 300, {"Trichy", "Tamil Nadu", "620001"}}
    };

    int searchCode, changeQty;

    printf("Enter product code to update: ");
    scanf("%d", &searchCode);

    printf("Enter quantity change (+ for incoming, - for outgoing): ");
    scanf("%d", &changeQty);

    // Get pointer to the matching product (no copy made)
    struct Product *ptr = searchProduct(inventory, MAX_PRODUCTS, searchCode);

    if (ptr != NULL) {
        printf("\nBefore update:\n");
        printf("Product: %s | Quantity: %d\n", ptr->productName, ptr->quantity);

        // Update using structure pointer - modifies original record directly
        updateStock(ptr, changeQty);

        printf("\nAfter update:\n");
        printf("Product: %s | Quantity: %d\n", ptr->productName, ptr->quantity);
    } else {
        printf("Product not found.\n");
    }

    return 0;
}
```

Run

Save

Input

stdin input

Output